EXPERIMENT 07

Component-Based Web Application using React JS

Roll no.- 16010125107

Name – Ishaan Singh Khanka

Course Outcome: CO4 – Understanding and implementing web application using React.

Aim: To develop a component-based web application using React JS by implementing JSX, functional components, props, state and event handling for case study on Climate Heatwave Monitoring.

Objectives

Create and run a basic React application using Node.js and npm.
Understand JSX as an XML-like syntax extension used inside JavaScript for describing React UI.
Develop reusable functional components.
Pass data between components using props.
Manage changing UI data using state and useState().
Handle user interaction using React event handlers.

NOTE: Refer to the "Case Study you selected" and complete the tasks in React. This write-up uses Student Information Application as a sample.

Software Requirements

Node.js and npm
Visual Studio Code
Google Chrome or another modern browser
Command Prompt/PowerShell

Theory

React and Existing HTML/CSS/JavaScript Knowledge

HTML provides structure, CSS provides presentation, and JavaScript provides programming logic and browser interaction. React organizes UI development around reusable components, JSX, props and state.

JSX and XML Correlation

JSX stands for JavaScript XML. It looks similar to XML/HTML because it uses angle-bracket tags, but JSX is not XML. It is a syntax extension used to describe React UI inside JavaScript and is transformed by the build tooling.

```
const name = "Riya";
function App() { return <h1>Hello {name}</h1>; }
```

HTML: class="box" → JSX: className="box".

HTML onclick="..." → JSX onClick={...}.

JSX permits JavaScript expressions inside { }.

XML represents structured data using tags; JSX describes UI.

JSON represents data using key-value pairs and arrays.

Functional Components

A functional component is a JavaScript function that returns JSX. Components divide a UI into reusable units.

```
function Student() {
  return <h2>Student Details</h2>;
}
```

Props

Props are read-only inputs supplied by a parent component to a child component.

```
function Student(props) {
  return <p>{props.name} - {props.course}</p>;
}

<Student name="Riya" course="B.Tech" />
```

State and Event Handling

State represents data that can change during application execution. Updating state causes React to render the relevant UI again.

```
const [age, setAge] = useState(20);
<button onClick={() => setAge(age + 1)}>Increase Age</button>
```

Problem Statement

Develop a Student Information application that displays student details using a reusable Student component. The application must accept student information through props and provide an interactive control that changes the displayed age using React state and event handling.

Tasks to be Completed:

1. Verify Node.js and npm installation.
2. Create a React application named react-student.
3. Run the application and identify the public and src folders.
4. Modify App.js to display a Student Information heading.
5. Create Student.js as a reusable functional component.
6. Pass name, course and age from App.js to Student.js using props.
7. Display the received information using JSX.

8. Add a state variable for age using useState().
9. Create an Increase Age button using onClick.
10. Apply basic CSS using App.css.
11. Run and test the application in the browser.
12. Capture screenshots of the final output and important source code.

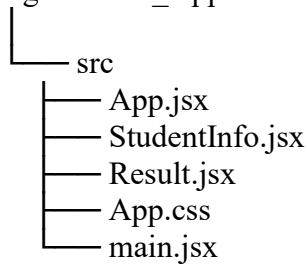
Procedure

13. Open Command Prompt and navigate to the practical folder, for example D:\React.
14. Run: npx create-react-app react-student
15. Run: cd react-student
16. Run: npm start
17. Open the project in VS Code.
18. Create Student.js inside src.
19. Write the Student component and export it.
20. Import Student into App.js and supply props.
21. Add useState and the Increase Age event.
22. Save and test the application.

Suggested Components

Keep the application very simple:

e.g. Student_Application



Program

// Student.js

```
import { useState } from "react";

function Student({ name, course, initialAge }) {
  const [age, setAge] = useState(initialAge);
  return (
    <div className="card">
      <h2>Student Details</h2>
      <p>Name: {name}</p>
      <p>Course: {course}</p>
```

```
      <p>Age: {age}</p>
      <button onClick={() => setAge(age + 1)}>
        Increase Age
      </button>
    </div>
  );
}
export default Student;
```

// App.js

```
import Student from "./Student";
import "./App.css";

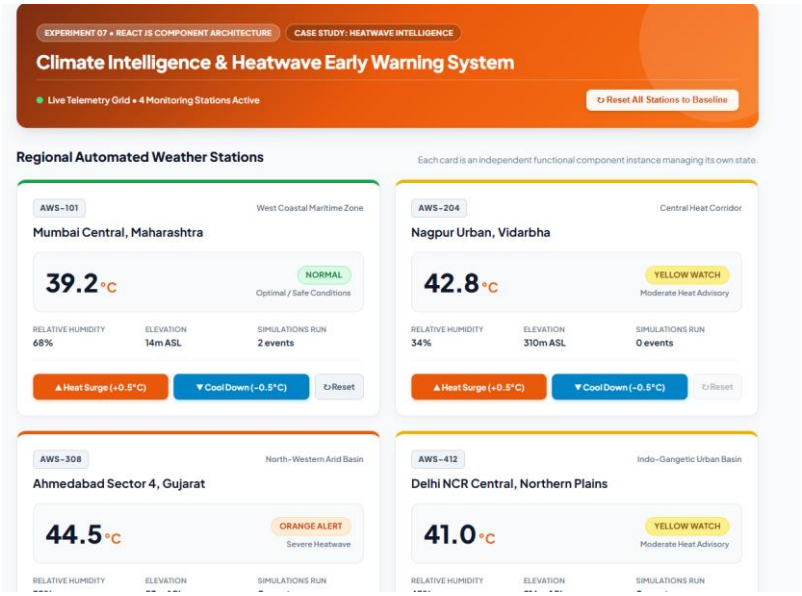
function App() {
  return (
    <div>
      <h1>Student Information</h1>
      <Student name="Riya" course="B.Tech" initialAge={20} />
    </div>
  );
}
export default App;
```

/* App.css */

```
.card { border: 1px solid #999; padding: 20px; width: 300px; margin: 20px auto; }
button { padding: 8px 15px; }
```

Expected Output

The browser displays student name, course, age and an Increase Age button. Each click increases the displayed age by one.



Result (Use your case Study)

The component-based Student Information application was developed successfully using JSX, functional components, props, state and event handling.

Post Lab Question:

1. How is JSX different from HTML?
2. What is a functional component?
3. Why is className used instead of class in JSX?
4. How is React different from direct DOM manipulation?